Dynamic Taint Analysis for the Detection of Sensitive Data Leaks

Anonymous authors
Anonymous institution
Anonymous city, State, Country

Abstract

In a world where IT systems are increasingly exposed to sensitive data leaks, these leaks can be caused by human error or malicious intent, originating from within or outside the organization. Numerous studies have proposed solutions, such as detection systems based on machine learning, approaches based on regex rules, and traditional SIEM platforms. These solutions enable the detection of certain sensitive values, but struggle to track their propagation in the execution context. In this article, we propose a Dynamic Taint Analysis approach designed to specifically detect sensitive data leaks. Our method tracks data flows marked as sensitive in a relational database and monitors their presence during exports to external files or via HTTP requests. The system analyzes the output generated by the database to produce event logs, which are then collected by the Wazuh SIEM, enriched by this mechanism, to trigger security alerts while filtering out non-sensitive occurrences to reduce false positives.

Through our experiments, we have been able to demonstrate that our approach effectively identifies data leaks, such as transfers from the database (source) to a file (destination) or from the database (source) to the network (destination), while maintaining real-time detection. This work highlights the potential of Dynamic Taint Analysis to improve the detection of sensitive data exfiltration. Unlike content-based detection, our approach treats sensitivity as a property of data provenance rather than textual form, enabling reliable detection even when syntactically identical values occur in both sensitive and non-sensitive datasets.

CCS Concepts

• **Security and privacy** → **Logic and verification**; • **Theory of computation** → *Formal languages and automata theory*.

Keywords

dynamic taint analysis, data leak detection, data provenance, sensitive data, SIEM, database exports, information flow tracking, false positive reduction

ACM Reference Format:
Anonymous authors. 2025. Dynamic Taint Analysis for the Detection of Sensitive Data Leaks. In *16th ACM Conference on Data and Application Security and Privacy*. ACM, New York, NY, USA, 12 pages. https://doi.org/0000001.0000001

1 Introduction

The rapid growth of data-driven systems has significantly increased the exposure of sensitive information to accidental leaks and malicious exfiltration. Modern information systems continuously manipulate large volumes of personal, financial, and operational data, which are often replicated across databases, exported to files, transmitted over networks, or logged for auditing and debugging purposes. As a result, sensitive values may leave their intended execution context through multiple heterogeneous channels, sometimes without explicit malicious intent. To mitigate this risk, organizations widely deploy Data Loss Prevention (DLP) mechanisms and Security Information and Event Management (SIEM) platforms. These systems typically rely on content-based techniques such as regular expressions, keyword matching, digital fingerprinting, or machine-learning models to identify sensitive data. While effective in detecting well-structured patterns such as credit card numbers or social insurance numbers, these approaches fundamentally treat sensitivity as a property of the data's textual form. Consequently, they are unable to determine whether a detected value is truly sensitive or merely syntactically similar to one.

This limitation becomes critical in practice, as many identifiers admit multiple legitimate interpretations. A numeric string may correspond to a phone number, a game passcode, an ISBN, or an industrial part number, depending entirely on the context in which it is used. Moreover, sensitive values may be deliberately embedded, encoded, or transformed within apparently benign payloads to evade signature-based detection. In such cases, syntactic inspection alone is insufficient to reliably characterize data leaks, leading to a large number of false positives and reducing the operational value of alerts. A key observation underlying this work is that sensitivity is not an intrinsic property of a value, but rather a property of its *provenance*. Whether a value should be considered sensitive depends on where it originates, how it is propagated, and through which channels it leaves the system. However, most existing detection techniques lack the ability to track instance-level provenance across heterogeneous components such as database servers, file systems, web servers, and SIEM pipelines. In this paper, we propose a provenance-aware Dynamic Taint Analysis approach for the detection of sensitive data leaks. Our system tracks concrete values originating from databases declared as sensitive and follows their propagation across SQL exports and HTTP-based channels, without requiring instrumentation of the database server or the monitored applications. By leveraging observable side channels such as query logs and web server logs, the system reconstructs data flows and associates each detected value with its source database. The outputs of this analysis are transformed into structured JSON events and integrated into the Wazuh SIEM platform, enabling real-time alerting, correlation, and visualization. Unlike traditional Dynamic Taint Analysis systems, which typically rely on binary taint semantics, our approach encodes source-specific provenance information, allowing it to distinguish syntactically identical values originating from sensitive and non-sensitive datasets.

We evaluate our approach in a controlled scenario involving two databases containing identical values but differing in sensitivity.

CODASPY '26, Frankfurt am Main, Germany
2025. ACM ISBN 123-4567-24-567/08/06
https://doi.org/0000001.0000001

Experimental results demonstrate that a regex-based detector inspired by Meli et al [24]. produces a high number of false positives in this setting, while our provenance-based Dynamic Taint Analysis system achieves perfect precision and recall. These results highlight the necessity of provenance-aware detection for reliable data leak prevention in modern, dynamic environments. In this work, we identify and formalize the limitations of content-based sensitive data detection in the presence of contextual and provenance ambiguity, design a provenance-aware Dynamic Taint Analysis system capable of tracking sensitive values across heterogeneous exfiltration channels without intrusive instrumentation, and integrate the proposed approach with the Wazuh SIEM, demonstrating its effectiveness through a comparative experimental evaluation.

2 Background

SIEM (Security Information and Event Management Systems) is a system designed to provide centralized management of security logs, from collection to analysis and storage. It aims to increase the level of information security, facilitate centralized log management and analysis, help organizations comply with various regulations (compliance often being the main driver for acquiring a SIEM), and reduce the risk of network intrusions [17]. SIEM combines the functionalities of security information management (SIM), which focuses on compliance, log storage, and policies, and security event management (SEM), which deals with real-time incident response and network security data. The fundamental operation of a SIEM can be described by Gartner's "five Cs": Collection, Consolidation, Correlation, Communication, and Control [17].

Collection: SIEM collects security log data from a large number of different devices and applications across the network. For sources that do not support standard collection protocols (such as Windows event logs), software called agents is installed on the source to translate and normalize the data before sending it. During this step, data can be filtered to reduce bandwidth and processing power requirements [17].

Consolidation: This process includes normalization (converting various formats into a proprietary format) and aggregation (grouping events of the same type). It is crucial to ensure that the time and date of all logs are correctly synchronized (for example, using the Network Time Protocol or NTP). Aggregation is important because it allows for the detection of intrusion attempts and suspicious actions that isolated messages would not reveal (e.g., numerous failed login attempts in quick succession).

Correlation: This is the analysis stage where different log events (of different types) are linked together to reconstruct a complete attack or incident. This process uses contextual information (knowledge base) about the specific network environment, such as user directories and device priority or location. The analysis can use anomaly-based detection or signature-based detection [17].

Communication: Following analysis, SIEM alerts administrators in real time or generates reports (regular or on demand). These alerts and reports inform staff and form part of incident management channels. Some SIEM products can even take protective or reactive measures (such as closing ports or removing malware) via connected devices [17].

Control: This involves data storage and archiving. Log data is retained, usually online for a short period of time, and then archived for compliance, forensic, and e-discovery purposes. To ensure the integrity of archived logs (essential as legal evidence), special security mechanisms, such as Write Once, Read Many (WORM) media, are often used to prevent the modification or destruction of written data [17].

There are many SIEM solutions available, which differ in terms of their various features and their ability to respond effectively to threats and prevent attacks. Among the existing solutions are IBM QRadar [14], which is a licensed SIEM, and Splunk SIEM. Other SIEMs include Exabeam [10] and LogRhythm [21]. However, there are other open-source solutions such as Wazuh [15], which offers advanced features such as integrity monitoring and is easy to integrate with any component. We can add to this list the open-source SIEMs Elastic Stack [25], Alienvault, and many other software programs that are capable of functioning as SIEM.

2.1 The Wazuh Security Platform

Wazuh is an open-source Security Information and Event Management (SIEM) and Host Intrusion Detection System (HIDS) platform widely adopted in industry for threat detection, compliance monitoring, vulnerability assessment, and real-time security analytics. Designed with modularity, extensibility, and scalability in mind, Wazuh offers a unified interface for collecting, processing, and analyzing security events across heterogeneous and highly distributed environments. These environments range from traditional on-premise endpoints and servers to virtualized infrastructures, cloud platforms, and container-orchestrated microservices architectures [31]. Its open-source licensing model, combined with rich integration capabilities and the absence of commercial licensing costs, positions Wazuh as a transparent and cost-effective alternative to proprietary SIEM systems such as Splunk Enterprise Security or IBM QRadar.

At the architectural level, Wazuh relies on an agent–server model. Lightweight agents deployed on monitored hosts continuously collect system-level telemetry, including file integrity changes, executed system calls, user and process activity, configuration modifications, authentication attempts, and network communications. The agents also support advanced host-based monitoring functions such as rootkit detection, vulnerability inventorying, and configuration assessment. Once collected, these events are securely transmitted to the Wazuh server, where they pass through a multi-stage processing pipeline involving decoding, field extraction, normalization, rule-based correlation, and enrichment with contextual metadata. The processed alerts are subsequently indexed into an Elasticsearch cluster and visualized through Kibana dashboards, providing analysts with powerful capabilities for real-time monitoring, correlation, and forensic investigation. This decoupled architecture allows Wazuh to scale efficiently from small deployments with a few agents to enterprise infrastructures generating millions of daily events.

A central component of Wazuh is its comprehensive rule engine. The rule engine applies thousands of predefined correlation rules spanning intrusion attempts, policy violations, system anomalies, malware indicators, suspicious authentication patterns, and unauthorized file access. These rules leverage both signature-based

mechanisms and heuristic analysis, using temporal patterns, contextual fields, and anomaly indicators to detect complex threat behaviors. Additionally, Wazuh provides native support for major compliance frameworks such as PCI-DSS, HIPAA, GDPR, CIS Benchmarks, and NIST 800-53 through dedicated auditing modules, automated configuration checks, and compliance reporting dashboards. As highlighted in [11], these capabilities position Wazuh among the most versatile SIEM platforms available for organizations seeking to strengthen visibility and reduce operational and regulatory risks [11].

Despite its extensive capabilities, Wazuh—like most SIEM systems—does not inherently determine whether the data involved in an event are *truly sensitive*, nor does it track how such values propagate across processes, files, databases, or communication channels. The platform excels at correlating logs and identifying suspicious activity, but it lacks the semantic context necessary to distinguish between benign data movement and actual sensitive data exfiltration.

Although Wazuh provides an extensive set of capabilities for collecting, correlating, and visualizing security events, its analytical model remains fundamentally centered on log observation and the interpretation of their immediate content. Detection mechanisms primarily rely on rules expressing syntactic relations, pattern matching, thresholds, or temporal correlations across events, which enables the effective identification of certain categories of attacks or anomalous behaviors. However, this approach does not preserve any explicit information about the origin of values throughout their lifecycle, nor about the paths taken by data as they move across different components of an information system. In practice, the SIEM processes each event as an essentially independent entity, enriched with local contextual metadata, but lacking instance-level provenance semantics.

This absence of provenance tracking becomes particularly problematic in sensitive data leak detection scenarios, where the sensitivity of a value cannot be inferred from its textual form alone. Identical numeric or alphanumeric identifiers may legitimately appear in databases of different nature, be exported into files, transmitted through HTTP requests, or recorded in application logs, without any reliable indication of their true origin in the observed content. In such contexts, a traditional SIEM cannot distinguish a value that actually originates from a sensitive source from an identical value originating from a non-sensitive dataset. This ambiguity leads to over-detection, the generation of poorly discriminative alerts, and ultimately a degradation of analysts' trust in deployed detection mechanisms.

This limitation is further amplified by the heterogeneous and distributed nature of modern environments, where data flows seamlessly across relational databases, file systems, web services, and logging pipelines. SIEM platforms, whose primary role is to aggregate and correlate events from these multiple sources, do not natively provide mechanisms to reconstruct inter-system data flows or to link an observed value back to its source database. As a result, while Wazuh excels at correlating events and identifying suspicious activity patterns, it cannot by itself provide the guarantees needed for reliable sensitive data leak detection in settings where provenance plays a decisive role.

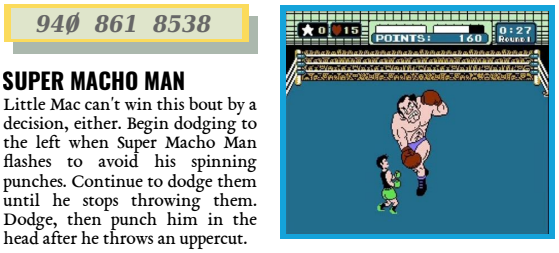


Figure 1: A passcode from *Mike Tyson's Punch-Out!*, which is also a valid US phone number.[1]

In this work, Wazuh is therefore used as a robust infrastructure for collecting, correlating, and visualizing alerts, while fine-grained data-flow analysis and provenance resolution are performed upstream by a Dynamic Taint Analysis engine. This complementarity preserves the operational benefits of an established SIEM while enriching security events with explicit provenance information, which is essential to distinguish real sensitive data leaks from benign occurrences and to significantly reduce false positives.

3 The Importance of Context

Sensitive values do not reveal their nature by their textual form. A sequence of digits or hexadecimal characters is considered sensitive only because of the *context in which it is interpreted*, and more fundamentally because of its *provenance*. In this section, we illustrate two complementary forms of ambiguity: (i) *accidental* double meanings, where a value legitimately appears in two unrelated domains, and (ii) *deliberate* or *constructed* double meanings, where a sensitive value is embedded or transformed within an apparently harmless payload. Both categories show that syntactic inspection alone is insufficient for determining whether a value constitutes a data leak.

3.1 Accidental Double Meanings

Certain digit or hexadecimal strings can legitimately appear in multiple, unrelated domains. When observing such values leaving a system, it is often impossible to determine, from their textual form alone, whether they correspond to sensitive information or to harmless data. We illustrate this ambiguity through representative examples.

US phone number and video-game passcode. The digit string `940 861 8538` is a valid US phone number, with area code `940` assigned to Texas. In another domain, however, the same value is used as a passcode in the Nintendo game *Mike Tyson's Punch-Out!!*, granting access to the penultimate opponent. Figure 1 shows the corresponding game screen.

Railway journey number and commit hash. The hexadecimal string `efecba` appears on certain SNCF railway tickets as a journey identifier (Fig. 2). The identical sequence also constitutes a valid Git short hash for a commit in a public repository.[2]

[1] Adapted from *Nintendo Players' Guide Top Secret Passwords*, 1992, p. 86. https://archive.org/details/Nintendo_Players_Guide_Top_Secret_Passwords_1992/page/85/mode/2up

[2] This commit exists at https://github.com/liflab/programmatic-spreadsheet/commit/efecba.

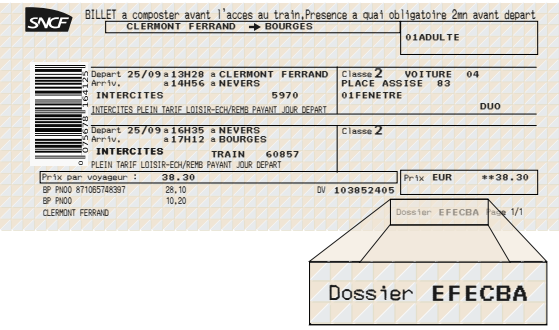


Figure 2: Illustration of an SNCF voyage number.

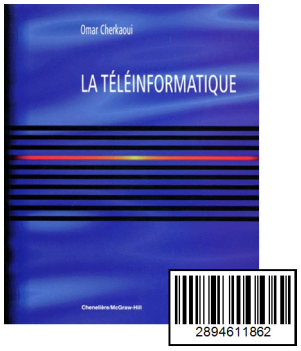
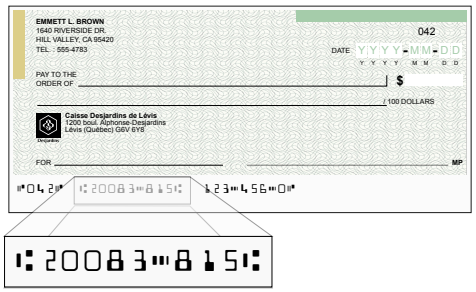
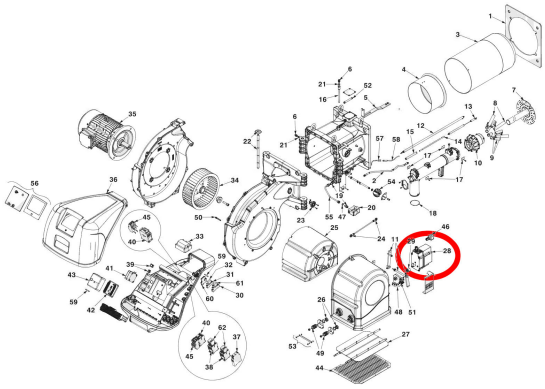


Figure 3: A book with the ISBN-10 number 2894611862.



(a) Desjardins branch number 20083815



(b) Riello part number 20083815 (in red)

Figure 4: A digit sequence with two unrelated interpretations.

Canadian phone number and ISBN-10. The ten-digit sequence `2894611862` corresponds both to a Canadian telephone number (prefix `289-461`, Bolton, Ontario) and to the ISBN-10 of an actual book, shown in Fig. 3 [8]. Its syntactic form alone does not indicate whether it encodes personal contact information or a bibliographic identifier.

Bank account prefix and industrial part number. The string `20083815` is simultaneously a valid branch–institution code for a Canadian bank account (Fig. 4a) and a part number for a Riello servomotor (Fig. 4b). Whether this value is sensitive or benign depends entirely on the domain in which it is interpreted.

These examples illustrate that, taken in isolation, the textual appearance of a value does not reliably indicate whether it is sensitive. Multiple, independently meaningful interpretations may coexist for a single sequence of digits or characters. The next subsection examines a complementary phenomenon in which sensitive values are *deliberately* embedded within innocuous-looking payloads.

3.2 Deliberate Double Meanings

The examples in the previous subsection arise from coincidental overlaps between different domains. However, modern systems increasingly encounter situations in which sensitive values are *deliberately* embedded, transformed, or concealed within larger payloads whose apparent meaning is innocuous. We refer to this phenomenon as *payload smuggling*. Unlike accidental ambiguity, payload smuggling exploits the malleability of data representations: a sensitive value may be encoded, renamed, embedded as a substring, or placed in a context where its sensitivity is not visually apparent.

Such techniques have long been documented in the security literature. Studies on data leak prevention (DLP) highlight how even simple re-formatting, encoding, or repackaging of content can defeat signature-based detection [27, 28]. Attacker taxonomies such as MITRE ATT&CK explicitly describe adversarial use of encoding, compression, and obfuscation to conceal sensitive or malicious content [1]. More broadly, the information-hiding and steganography literature demonstrates the feasibility of embedding confidential data within apparently benign structures [16, 26]. In all these cases, the visible syntax of a payload does not reliably reveal whether it carries sensitive information.

We illustrate this phenomenon through representative patterns commonly observed in operational systems.

Embedding as a numeric field. A sensitive identifier may be injected into a field whose semantics suggests a non-sensitive interpretation. For example, a ten-digit phone number may appear in an application log as an execution time:

```
INFO completed request in 9408618538 ms
```

Although the substring resembles a duration, its origin may lie in a confidential `customer.phone` column. Content-based detection cannot distinguish between the two.

Sensitive values hidden in filenames. Filenames and object identifiers are frequently used as carriers for hidden data, intentionally or accidentally. A social insurance number may, for example, appear inside a filename:

```
report_987654321.csv
```

Such values are routinely interpreted as counters, timestamps, or arbitrary numeric identifiers by automated systems.

Encoding to evade pattern-based detection. Encoding techniques such as Base64 or hexadecimal conceal the surface structure of sensitive data while leaving its meaning intact. An HTTP request of the form

```
GET /export?token=NTUyMjMzNDU=
```

may embed a sensitive identifier that no longer matches any regular-expression pattern. Encoding as an evasion vector is well documented in both malware analysis and DLP studies [1, 2].

Embedding inside structured data. Sensitive attributes may be inserted into JSON, XML, or other structured payloads where they resemble harmless values. In a REST API, for example, a value such as:

```
"id": "112233445"
```

may either be a user-generated identifier or a leaked social insurance number. Such smuggling is particularly difficult to detect in logs that aggregate mixed-provenance data from multiple sources.

Using external formats as carriers. Data may also be concealed within media or document formats. Although our focus is on database and network flows, it is worth noting that polyglot files and steganographic carriers allow one physical payload to satisfy multiple interpretations simultaneously [16, 19]. A seemingly benign image or PDF file may embed sensitive strings without any syntactic indication of their presence.

Polyglot files as a general case. A particularly striking illustration of deliberate ambiguity is the *polyglot file*: a single byte sequence that simultaneously satisfies multiple file-format specifications (for example, a valid PDF document that is also a valid ZIP archive or JavaScript program). Polyglot constructions have been widely documented in security research [19]; they enable a payload to appear harmless to one parser while revealing a completely different meaning to another. In such cases, the bytes do not reveal whether the file should be interpreted as benign or harmful: the outcome depends entirely on the *context of interpretation*—namely, which subsystem opens the file and what assumptions it makes about its provenance. Polyglot files therefore exemplify the broader challenge discussed in this section: the same raw data may have multiple legitimate interpretations, and syntactic inspection alone cannot determine which one is relevant for security purposes.

These examples illustrate that sensitive data may circulate through channels where it appears as an innocuous substring, an encoded value, or a field with an unrelated apparent meaning. Existing DLP and signature-based techniques are known to be vulnerable to such transformations [27, 28]. Ultimately, whether the value is sensitive cannot be determined from its textual form alone. Its sensitivity depends on where it *originates* and how it is *propagated*. The remainder of this paper develops a provenance-based detection model designed to address precisely this challenge. These examples motivate the need for detection mechanisms that reason about provenance rather than surface syntax.

4 Related Work

A large body of work has addressed the detection of sensitive data leaks using syntactic, semantic, or behavioral analyses. Although these approaches differ in methodology, we shall see that they share a fundamental limitation: they treat sensitivity as a *property of the content*, not as a *property of the data's origin*.

4.1 Signature- and Regex-Based Detection

Regular expressions and keyword-based filtering remain the most widely deployed techniques in commercial DLP tools [6, 18]. These approaches detect fixed numeric or textual patterns (e.g., credit-card formats, SSNs), often combined with simple heuristics. Although inexpensive and easy to deploy, pattern-based systems examine only the *surface form* of the data. They therefore cannot differentiate between a sensitive value and an unrelated identifier that happens to share its syntactic format, resulting in high false positives—a phenomenon extensively documented in prior work such as Meli et al. [24].

4.2 Digital Fingerprinting

Digital fingerprinting techniques construct signatures of confidential documents or data corpora using n-grams, k-skip-n-grams, or fuzzy fingerprints [29, 30]. These methods can detect rephrased or partially transformed content, but they identify similarity only with respect to a pre-registered confidential dataset. Fingerprinting thus collapses in scenarios where two independent sources legitimately contain identical values: the detector can match the string, but cannot determine *which* source it came from. As a result, fingerprinting cannot differentiate between a sensitive identifier and an identical identifier stored in a public or benign dataset.

4.3 Machine-Learning Approaches

Machine-learning-based DLP systems classify data according to topic-level semantics or contextual features [3, 5, 22]. When sensitive information appears in rich natural-language environments, ML models can exploit lexical or distributional patterns surrounding identifiers (e.g., words such as "SSN", "account", or "diagnosis"). However, most operational leaks involve identifiers exported via SQL, OUTFILE, HTTP parameters, filenames, or log messages—contexts that offer little or no linguistic structure. In these settings, an ML model receives the same isolated token for both sensitive and non-sensitive values and therefore lacks a discriminative basis for classification. Moreover, ML encodes *semantic* similarity, not *provenance*, and thus cannot determine whether an identifier originated from a protected database or from a benign dataset.

4.4 Static Analysis

Static taint analysis tools examine program code (e.g., Android APKs or server-side sources) to identify potential source–sink flows [32, 33]. This approach effectively detects leaks that are expressible in the application's control and data-flow graphs. However, static analysis assumes visibility into program structure and does not

observe data flows that occur outside the program boundary, such as database exports, file writes, or HTTP logs. It also cannot distinguish between different runtime databases that contain identical values, since it has no model of instance-level data provenance.

4.5 Traditional Dynamic Taint Analysis

Dynamic Taint Analysis, also known as dynamic information flow tracking, is the preferred approach for analyzing software security by monitoring contaminated data during program execution (runtime) [4, 20, 34]. The fundamental concept of Dynamic Taint Analysis is to label certain incoming data values as tainted, particularly those introduced from sensitive or untrusted sources (the source), such as application APIs or network interfaces. These contamination labels are then propagated as instructions are executed, which involves marking values in CPU registers or memory locations [4]. A contamination propagation policy is associated with each instruction to determine whether output operands should inherit the contamination status of input operands.

Classical DTA can detect explicit flows leaving a program, but it typically provides only binary taint semantics ("tainted" vs. "untainted") and does not encode which sensitive source introduced the taint. Moreover, traditional DTA operates within a single instrumented program and cannot track flows that span heterogeneous systems—such as database servers, MySQL OUTFILE operations, webserver logs, and SIEM ingestion pipelines. As a result, standard DTA cannot distinguish identical values originating from different databases unless augmented with higher-level provenance semantics.

4.6 Discussion

Table 1 summarizes the fundamental limitations shared by content-based and program-centric DLP techniques. Despite their methodological diversity, all prior approaches ultimately rely on observable properties of the *data itself* (its syntax, statistics, or semantics). None is equipped to track the *origin* of a value across heterogeneous execution contexts. Consequently, they cannot differentiate two identical values drawn from sensitive and non-sensitive datasets, nor can they reliably detect values that have been encoded, embedded, or smuggled inside other payloads.

5 A Provenance-Based Monitoring Approach

The limitations of existing approaches motivate the development of a *provenance*-centered model. Provenance, also called *lineage*, is defined as the process of relating the outputs of a data processing task to its inputs [13]. Under this broad acception, several lines of work can be considered related to provenance. This includes data lineage in database queries [7, 9, 12], or taint analysis for the detection of information leaks in a system [23].

5.1 Design Requirements

Our design is guided by three requirements:

- **Provenance awareness.** The detector must distinguish identical values that originate from different sources (e.g., two databases containing overlapping data), and must propagate this information across heterogeneous channels (file exports, HTTP requests, logs).
- **Non-intrusiveness.** The detector should operate without modifying the database server or the monitored applications. It must rely on observable side channels such as query logs, export commands, and webserver logs.
- **SIEM integration.** The system should emit structured alerts that can be ingested by an existing SIEM (here, Wazuh), which is then responsible for storage, correlation, and visualization.

5.2 Architecture Overview

Figure 5 presents an overview of the monitoring architecture. The central component is a Python-based Dynamic Taint Analysis (DTA) engine that observes database activity and export channels, resolves provenance for exported values, and emits JSON alerts to be collected by a Wazuh agent.

At a high level, the system consists of the following elements:

- **Configuration table (`dta_config`).** A dedicated configuration table in the database declares which databases or tables are considered sensitive. For each monitored database, `dta_config` stores an identifier and a Boolean flag indicating whether it is sensitive.
- **Taint pool.** At startup and periodically thereafter, the DTA engine connects to the database and loads all values from the declared sensitive databases into an in-memory *taint pool*. Conceptually, this pool associates each concrete value with a provenance label indicating the sensitive source(s) from which it originates.
- **Log and export monitors.** The engine monitors the MySQL general query log to observe SQL activity (including `SELECT` and `OUTFILE` statements) and parses Apache HTTP logs to observe network exports. These sources are used to reconstruct how values move from the database to files or HTTP responses.
- **Provenance resolver.** For each observed export operation, a provenance resolver attempts to determine which database is the origin of the exported content, based on a combination of log information and query parameters.
- **Alert emitter.** When an exported value matches the taint pool and its origin is a database declared as sensitive, the engine emits a structured JSON alert describing the leak. These JSON records are written into a directory monitored by a Wazuh agent, which forwards them to the SIEM.

5.3 Provenance Resolution Logic

The core of the system lies in resolving the provenance of exported data without instrumenting the database server. We consider two main exfiltration channels in our prototype: SQL exports using the `OUTFILE` command and HTTP-based exports.

File exports via `OUTFILE`.. When an `OUTFILE` command is issued, the database server writes query results directly to a file. The DTA engine observes this operation through the general log and maintains a small *OUTFILE mapping* of the form

$$\texttt{outfile_provenance} : \texttt{file} \mapsto (\texttt{database}, \texttt{timestamp}).$$

When the engine later reads the contents of that file to search for tainted values, it uses this mapping to associate any matches with

Table 1: Comparison of leak-detection approaches with respect to provenance awareness

Approach	Content-Based	Provenance-Aware	Core Limitation
Regex / Signatures	Yes	No	Cannot distinguish syntactically identical sensitive and non-sensitive values.
Fingerprinting	Yes	No	Matches content similarity only; fails when two sources contain identical values.
Machine Learning	Yes (semantic)	No	Cannot classify isolated identifiers or infer source database; no instance-level provenance.
Static Analysis	Partial	No	Cannot observe DB exports, logs, or multi-system flows; lacks runtime source identity.
Traditional DTA	Partial (byte-level)	No	Tracks taint but not source-specific provenance; cannot follow cross-system flows.
Our Approach (Provenance DTA)	No	**Yes**	Tracks origin across heterogeneous systems; distinguishes identical values by provenance.

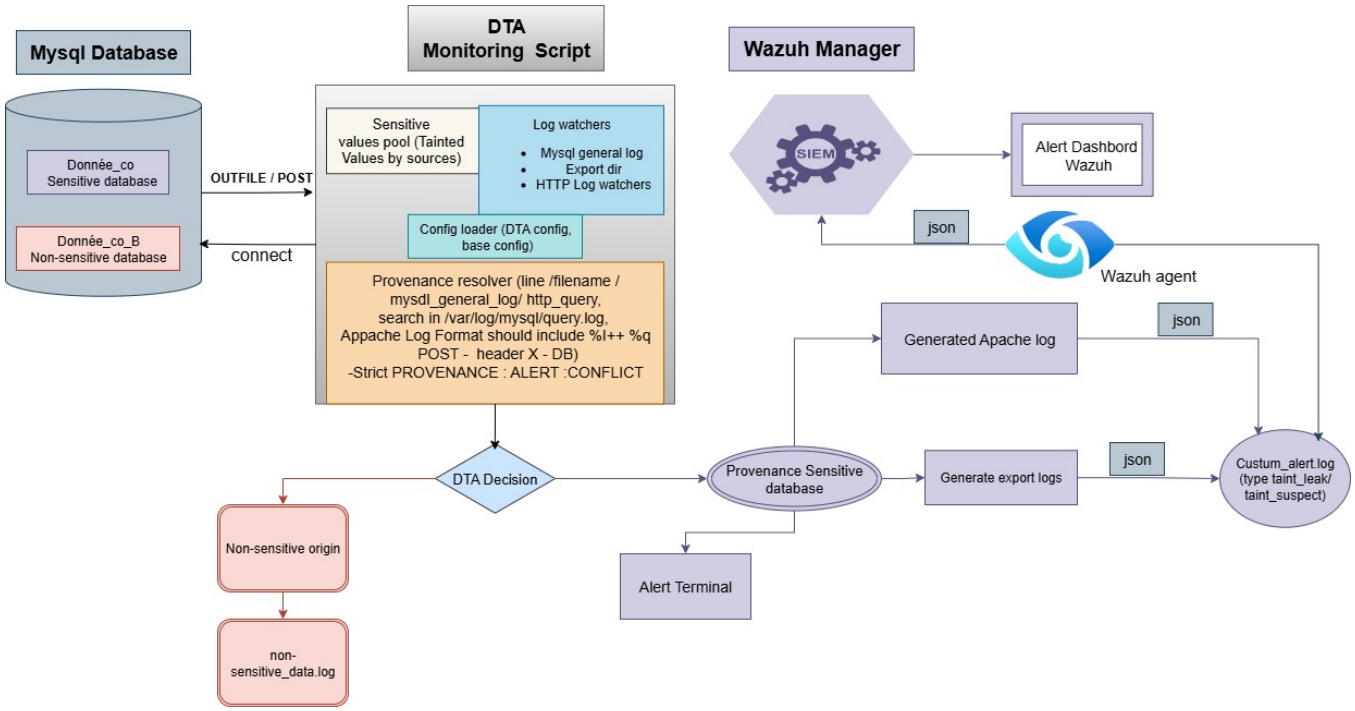


Figure 5: Overview of the provenance-based Dynamic Taint Analysis architecture integrated with the Wazuh SIEM.

the database that produced the file. To handle concurrency and logging delays, the mapping is updated with a short grace period (e.g., 50 ms retries within a 300 ms window), and entries are given a time-to-live to bound memory usage.

Network exports via HTTP.. For HTTP-based exports, the engine inspects Apache logs and reconstructs the relevant requests, including URL parameters such as `?db=xxx`. When a value found in an HTTP response also appears in the taint pool, the engine uses the parameters of the corresponding request to infer the most likely origin database. If multiple inference methods (OUTFILE mapping, HTTP parameters, log context) disagree, the engine records a provenance conflict and can optionally raise a lower-severity `taint_suspect` alert.

Matching against the taint pool. Once a provenance label has been assigned to an export event, the engine scans the exported payload (file contents or HTTP response) for exact matches with values in the taint pool. If a match is found and the associated provenance is marked sensitive in `dta_config`, the event is classified as a confirmed leak. If no provenance can be determined, or if provenance is inconsistent, the event is either logged as non-sensitive or flagged as suspicious depending on configuration flags such as `ALERT_ON_UNKNOWN_PROVENANCE`.

5.4 Alert Generation and SIEM Integration

For each confirmed leak, the DTA engine emits a JSON record containing:

- the detected value;
- the event type (e.g., `taint_leak` or `taint_suspect`);
- the inferred provenance (database name, and optionally table);
- the exfiltration channel (`EXPORT`, `HTTP`, etc.);
- timestamps and correlation identifiers.

These records are written to a directory monitored by a Wazuh agent. On the SIEM side, a small set of custom Wazuh rules matches these JSON events, assigns appropriate severity levels, and indexes them into a dedicated alert index. Figure 10 illustrates how the resulting alerts appear in the Kibana/Discover dashboard. The SIEM is thus used as a standard correlation and visualization backend; the novelty of our contribution lies in the provenance-based classification performed upstream by the DTA engine.

6 Evaluation Scenario: Identical Sensitive and Non-Sensitive Databases

To exercise the provenance-based detector in a controlled yet realistic setting, we constructed a scenario with two relational databases that contain overlapping values but differ in sensitivity. One database is declared sensitive (e.g., customer records), while the other is declared non-sensitive (e.g., test data, public samples). The goal is to verify that our DTA engine raises alerts only when values originate from the sensitive database, even when the same values appear in both.

```
mysql> SHOW TABLES;
+-----------------------+
| Tables_in_dta_config |
+-----------------------+
| bases_config          |
+-----------------------+
1 row in set (0,01 sec)

mysql> SELECT * FROM bases_config;
+--------------+----------+
| base_name    | sensible |
+--------------+----------+
| donnees_co   |        1 |
| donnees_co_B |        0 |
+--------------+----------+
2 rows in set (0,08 sec)

mysql>
```

Figure 6: Example of `dta_config` table declaring sensitive and non-sensitive databases.

6.1 Scenario Setup

A configuration table `dta_config` declares which databases are sensitive. Figure 6 illustrates a simple configuration in which one database is marked sensitive and another non-sensitive.

For the purpose of the experiment, the two databases are populated with identical records (e.g., social insurance numbers with the prefix `NAS`). From a content perspective, any export from either database looks the same. Only the provenance label stored in `dta_config` distinguishes sensitive from non-sensitive flows.

6.2 Exports from the Sensitive Database

We first issue SQL export operations from the sensitive database using the `OUTFILE` command and through an HTTP endpoint. In both cases, the DTA engine:

(1) observes the export through the MySQL general log or Apache log;
(2) resolves the origin database using the provenance resolver;
(3) scans the exported payload for values present in the taint pool;
(4) emits a `taint_leak` JSON alert for each match.

Figure 7 shows a representative case where five sensitive values are exported to a text file; each value is associated with a JSON alert indicating the detected value, the sensitive database of origin, the export channel, and the provenance resolution method.

A similar behaviour is observed for HTTP-based exports, where the DTA engine reconstructs the request parameters and associates the leaked value with the sensitive database selected in the URL.

6.3 Exports from the Non-Sensitive Database

We then repeat the same export operations from the non-sensitive database, which contains the same values. From the perspective of a regex-based detector, these exports are indistinguishable from the previous ones: they contain strings that match the pattern of a social insurance number. However, our DTA engine classifies them differently because their provenance label is non-sensitive.

Figure 8 shows that, in this case, the engine logs the presence of the values but does not emit `taint_leak` alerts. Instead, events are recorded as informational detections indicating that the values originate from a non-sensitive database.

```
🟢 Mapping OUTFILE : FILE_SENSITIVE_DATABASE.txt ← DB=donnees_co_B (thread 223)
⚠ Détection non sensible: {'timestamp': '2025-10-27T21:38:40.019139', 'valeur_detectee': 'NAS123456789', 'canal': 'EXPORT', 'fichier': 'FILE_SENSITIVE_DATABASE.txt', 'provenance_method': 'mysql_general_log', 'provenance_db': 'donnees_co_B', 'provenance_conflict': False}
⚠ Détection non sensible: {'timestamp': '2025-10-27T21:38:40.073186', 'valeur_detectee': 'NAS987654321', 'canal': 'EXPORT', 'fichier': 'FILE_SENSITIVE_DATABASE.txt', 'provenance_method': 'mysql_general_log', 'provenance_db': 'donnees_co_B', 'provenance_conflict': False}
⚠ Détection non sensible: {'timestamp': '2025-10-27T21:38:40.117613', 'valeur_detectee': 'NAS112233445', 'canal': 'EXPORT', 'fichier': 'FILE_SENSITIVE_DATABASE.txt', 'provenance_method': 'mysql_general_log', 'provenance_db': 'donnees_co_B', 'provenance_conflict': False}
⚠ Détection non sensible: {'timestamp': '2025-10-27T21:38:40.161922', 'valeur_detectee': 'NAS556677889', 'canal': 'EXPORT', 'fichier': 'FILE_SENSITIVE_DATABASE.txt', 'provenance_method': 'mysql_general_log', 'provenance_db': 'donnees_co_B', 'provenance_conflict': False}
⚠ Détection non sensible: {'timestamp': '2025-10-27T21:38:40.251172', 'valeur_detectee': 'NAS998877665', 'canal': 'EXPORT', 'fichier': 'FILE_SENSITIVE_DATABASE.txt', 'provenance_method': 'mysql_general_log', 'provenance_db': 'donnees_co_B', 'provenance_conflict': False}
```

Figure 7: Real-time detection of sensitive values exported from a database declared as sensitive.

```
🟢 Mapping OUTFILE : FILE_SENSITIVE_DATABASE.txt ← DB=donnees_co_B (thread 223)
⚠ Détection non sensible: {'timestamp': '2025-10-27T21:38:40.019139', 'valeur_detectee': 'NAS123456789', 'canal': 'EXPORT', 'fichier': 'FILE_SENSITIVE_DATABASE.txt', 'provenance_method': 'mysql_general_log', 'provenance_db': 'donnees_co_B', 'provenance_conflict': False}
⚠ Détection non sensible: {'timestamp': '2025-10-27T21:38:40.073186', 'valeur_detectee': 'NAS987654321', 'canal': 'EXPORT', 'fichier': 'FILE_SENSITIVE_DATABASE.txt', 'provenance_method': 'mysql_general_log', 'provenance_db': 'donnees_co_B', 'provenance_conflict': False}
⚠ Détection non sensible: {'timestamp': '2025-10-27T21:38:40.117613', 'valeur_detectee': 'NAS112233445', 'canal': 'EXPORT', 'fichier': 'FILE_SENSITIVE_DATABASE.txt', 'provenance_method': 'mysql_general_log', 'provenance_db': 'donnees_co_B', 'provenance_conflict': False}
⚠ Détection non sensible: {'timestamp': '2025-10-27T21:38:40.161922', 'valeur_detectee': 'NAS556677889', 'canal': 'EXPORT', 'fichier': 'FILE_SENSITIVE_DATABASE.txt', 'provenance_method': 'mysql_general_log', 'provenance_db': 'donnees_co_B', 'provenance_conflict': False}
⚠ Détection non sensible: {'timestamp': '2025-10-27T21:38:40.251172', 'valeur_detectee': 'NAS998877665', 'canal': 'EXPORT', 'fichier': 'FILE_SENSITIVE_DATABASE.txt', 'provenance_method': 'mysql_general_log', 'provenance_db': 'donnees_co_B', 'provenance_conflict': False}
```

Figure 8: Exports containing identical values from a non-sensitive database are logged but do not trigger leak alerts.

6.4 SIEM Visualization

Finally, we verify that the alerts generated by the Dynamic Taint Analysis engine are correctly integrated into the SIEM infrastructure and presented to analysts in a clear and interpretable manner. Figure 10 illustrates the Wazuh/Kibana Discover interface populated with alerts produced by the DTA system. Each row displayed in the interface corresponds to a confirmed taint_leak event emitted by the DTA engine and ingested by Wazuh without additional processing. These alerts are indexed in Elasticsearch and made available for inspection through the standard SIEM visualization workflow.

Each alert is enriched upstream by the Dynamic Taint Analysis engine with explicit metadata describing the detected leak. In particular, the alert records include the identifier of the detection rule, the assigned severity level, the detected value, the provenance resolution method, the inferred source database, and the exfiltration channel through which the value was exposed. This information is displayed directly in the SIEM interface, allowing analysts to understand the nature and origin of each detected event without requiring manual reconstruction or heuristic interpretation within the SIEM.

The visualization further confirms that no corresponding taint_leak entries are generated for exports originating from the non-sensitive database, even when the exported values are syntactically identical to those contained in the sensitive database. Instead, such exports are either ignored or recorded as non-sensitive detections, depending on the configuration, and do not appear as confirmed leak alerts in the SIEM dashboard. This behavior is directly observable in the Discover view, where all displayed alerts correspond exclusively to provenance-resolved leaks.

This scenario illustrates that the Dynamic Taint Analysis engine enforces a strictly provenance-based notion of sensitivity. A value that appears in both sensitive and non-sensitive databases is classified as a leak only when its inferred provenance points to a database declared as sensitive in the dta_config table. The SIEM plays no role in this decision-making process and merely reflects the detection results produced by the Dynamic Taint Analysis system. This behavior contrasts sharply with content-based detectors, which necessarily generate identical alerts for both types of exports due to their reliance on syntactic matching alone, as further quantified in the following section.

Overall, the SIEM visualization confirms that provenance-aware detection can be operationalized using standard SIEM tooling, while preserving an unambiguous interpretation of sensitive data leaks driven entirely by the external Dynamic Taint Analysis engine.

Metric	Meli System (Regex)	DTA System
True Positives (TP)	399	400
False Positives (FP)	397	0
False Negatives (FN)	1	0
True Negatives (TN)	3	0
Precision (%)	50.1%	100%
Recall (%)	99.75%	100%
F1-score (%)	66.7%	100%

Table 2: Performance comparison between a regex-based detector inspired by Meli et al. [24] and our provenance-aware Dynamic Taint Analysis (DTA) system.

6.5 Comparison with a Regex Baseline

We evaluated our provenance-based DTA engine against a content-based detector constructed from the regular-expression rules of Meli et al. [24]. Regex-based detection remains the de facto standard in deployed DLP solutions and SIEM rule-sets [6, 18], and Meli et al. provide a representative academic instance of this family. In contrast, more sophisticated approaches based on machine learning or advanced fingerprinting rarely provide publicly available implementations, and reproducing their models from a textual description alone would require re-creating training pipelines and tuning datasets, raising serious questions of fairness and comparability. For this reason, we restrict the empirical comparison to a regex baseline that we could faithfully reimplement from the published patterns.

input.type: log agent.ip: 172.21.2.51 agent.name: amadou agent.id: 001 manager.name: Amadou data.fichier: FILE_SENSITIVE_DATABASE.txt data.valeur: NAS998877665 data.provenance_conflict: false data.label: valeur_sensible data.canal: EXPORT data.source: donnees_co data.provenance_method: mysql_general_log data.type: taint_leak data.timestamp: Oct 27, 2025 @ 16:58:00.421 rule.firedtimes: 19 rule.mail: true rule.level: 12 rule.description: 🚨 Fuite de donnée sensible (EXPORT ou HTTP) détectée par DTA rule.groups: local, syslog, sshdtaint, dta, export, network rule.id: 110000 location: /var/ossec/logs/custom_alerts.log decoder.name: json id: 1761598680.4377161 full_log: {"timestamp":"2025-10-27T20:58:00.421998","type":"taint_leak","valeur":"NAS998877665","label":"valeur_sensible","canal":"EXPORT","source":"donnees_co","provenance_method":"mysql_general_log","provenance_conflict":false,"fichier":"FILE_SENSITIVE_DATABASE.txt"} timestamp: Oct 27, 2025 @ 16:58:00.816 _index: wazuh-alerts-

input.type: log agent.ip: 172.21.2.51 agent.name: amadou agent.id: 001 manager.name: Amadou data.fichier: FILE_SENSITIVE_DATABASE.txt data.valeur: NAS556677889 data.provenance_conflict: false data.label: valeur_sensible data.canal: EXPORT data.source: donnees_co data.provenance_method: mysql_general_log data.type: taint_leak data.timestamp: Oct 27, 2025 @ 16:58:00.377 rule.firedtimes: 18 rule.mail: true rule.level: 12 rule.description: 🚨 Fuite de donnée sensible (EXPORT ou HTTP) détectée par DTA rule.groups: local, syslog, sshdtaint, dta, export, network rule.id: 110000 location: /var/ossec/logs/custom_alerts.log decoder.name: json id: 1761598680.4376423 full_log: {"timestamp":"2025-10-27T20:58:00.377478","type":"taint_leak","valeur":"NAS556677889","label":"valeur_sensible","canal":"EXPORT","source":"donnees_co","provenance_method":"mysql_general_log","provenance_conflict":false,"fichier":"FILE_SENSITIVE_DATABASE.txt"} timestamp: Oct 27, 2025 @ 16:58:00.814 _index: wazuh-alerts-

Figure 9: Visualization of provenance-based leak alerts in the Wazuh Discover dashboard.

t agent.id	001
t agent.ip	172.21.2.51
t agent.name	amadou
t data.canal	EXPORT
t data.fichier	FILE1_SENSITIVE_DATABASE6.txt
t data.label	valeur_sensible
t data.provenance_conflict	false
t data.provenance_method	mysql_general_log
t data.source	donnees_co
data.timestamp	Oct 27, 2025 @ 16:52:03.332
t data.type	taint_leak
t data.valeur	NAS987654321
t decoder.name	json
t full_log	{"timestamp":"2025-10-27T20:52:03.332507","type":"taint_leak","valeur":"NAS987654321","label":"valeur_sensible","canal":"EXPORT","source":"donnees_co","provenance_method":"mysql_general_log","provenance_conflict":false,"fichier":"FILE1_SENSITIVE_DATABASE6.txt"}
t id	1761598324.4350308
t input.type	log
t location	/var/ossec/logs/custom_alerts.log
t manager.name	Amadou
t rule.description	🚨 Fuite de donnée sensible (EXPORT ou HTTP) détectée par DTA
# rule.firedtimes	11
t rule.groups	local, syslog, sshdtaint, dta, export, network

Figure 10: Details on the sensitive data leak alert

Our experiment is designed to stress precisely the scenario in which content-based detectors are structurally limited: identical values appearing in both sensitive and non-sensitive databases. We executed a set of controlled SQL export queries including 400 truly sensitive values (social insurance numbers) from a database declared as sensitive (`donnees_co`). In parallel, we introduced syntactically similar values into a non-sensitive database (`donnees_co_B`) and issued 400 export queries from `donnees_co_B`. The two databases thus contain overlapping values that are indistinguishable at the syntactic level but differ in provenance.

Results are summarized in Table 2. As expected, the regex-based detector successfully identified almost all sensitive values in exports from `donnees_co`, with 399 true positives (TP) and 1 false negative (FN). However, when faced with exports from the non-sensitive database `donnees_co_B`, the same detector produced 397 false positives (FP) and only 3 true negatives (TN). This behaviour is not a defect of the implementation: it directly follows from the fact that the regex rules operate solely on the textual structure of the values and have no access to their origin. Any value matching the social-insurance-number pattern is treated as sensitive, regardless of whether it comes from a confidential table or from a benign dataset containing, for example, game passcodes or boiler part numbers.

In contrast, our DTA system achieved perfect precision and recall on the same set of 800 exports: it correctly raised alerts for all 400 exports involving values from the sensitive database and raised none for exports involving only the non-sensitive database. This result follows from two design choices. First, sensitivity is associated with the provenance of a value—whether it originates from a database declared as sensitive in the `dta_config` table—rather than with its textual form. Second, the DTA engine tracks the propagation of these provenance labels across export channels (OUTFILE, HTTP) and correlates each detected value with its source database before deciding whether to alert.

It is important to emphasize that this experiment is not intended to show that regex-based methods are "wrong" or "bad" in general. In conventional scenarios where sensitive values appear only in protected datasets, regex detectors behave as expected and can achieve high F1-scores, as our true-positive and false-negative counts on `donnees_co` confirm. Instead, the experiment quantifies the extent to which content-only detection deteriorates in the presence of ambiguous values that legitimately occur in both sensitive and non-sensitive sources. These are precisely the situations illustrated in Section 3, and they are fundamentally beyond the reach of syntax-based detection. The provenance-aware DTA model is designed to remain robust under such ambiguity by construction.

7 Conclusion

This work demonstrates that Dynamic Taint Analysis provides an effective and reliable approach for detecting sensitive data leaks in real time. Unlike traditional methods based on regular expressions, content analysis, or machine-learning models, our approach relies on data provenance rather than on syntactic patterns alone. By explicitly tracking sensitivity markers and correlating each detected value with its source database and exfiltration channel, the DTA engine successfully distinguishes legitimate operations from genuine data leaks. This provenance-centered model drastically reduces false positives and eliminates false negatives, as confirmed by our experimental evaluation comparing DTA with the regular-expression system of Meli et al [24]. Our prototype integrates seamlessly with the Wazuh SIEM, enabling automatic generation, normalization, and visualization of alerts within a unified monitoring infrastructure. Experiments conducted across multiple exfiltration vectors including SQL exports via the `OUTFILE` command and HTTP requests demonstrate that the system maintains accurate detection across heterogeneous channels while preserving real-time processing capabilities. Moreover, the ability to correctly classify syntactically identical values based on their provenance context highlights the practical value of provenance-driven detection in modern, dynamic, and multi-source environments.

Overall, these results underscore the relevance of Dynamic Taint Analysis as a robust foundation for preventing sensitive data leaks. Future work includes extending the approach to more complex propagation paths, improving scalability for large-scale deployments, and integrating automated response mechanisms to further strengthen organizational security. More broadly, our results support a provenance-centric view of data protection: in a world where the same string can be an ISBN, a phone number, a bank identifier, or a boiler part number, the only robust way to decide whether an occurrence is sensitive is to track where it came from and how it flowed through the system.

References

[1] Mitre att&ck technique t1027: Obfuscated files or information. https://attack.mitre.org/techniques/T1027/, 2020.

[2] How base64 encoding opens the door for malware. https://www.opswat.com/blog/how-base64-encoding-opens-the-door-for-malware, 2021.

[3] T. Al-Shehari and R. A. Alsowail. An insider data leakage detection using one-hot encoding, synthetic minority oversampling and machine learning techniques. *Entropy*, 23(10):1258, 2021.

[4] A. M. Alashjee, S. Duraibi, and J. Song. Dynamic taint analysis tools: A review. *International Journal of Computer Science and Security (IJCSS)*, 13(6):231–244, 2019.

[5] S. Alneyadi, E. Sithirasenan, and V. Muthukkumarasamy. Detecting data semantic: a data leakage prevention approach. In *2015 IEEE Trustcom/BigDataSE/ISPA*, volume 1, pages 910–917. IEEE, 2015.

[6] T. Brindha and R. Shaji. An analysis of data leakage and prevention techniques in cloud environment. In *2015 International Conference on Control, Instrumentation, Communication and Computational Technologies (ICCICCT)*, pages 350–355. IEEE, 2015.

[7] P. Buneman, S. Khanna, and W. C. Tan. Why and where: A characterization of data provenance. In *Proc. ICDT 2001*, pages 316–330, 2001.

[8] O. Cherkaoui. *La téléinformatique.* Chenelière-McGraw Hill, 2003.

[9] Y. Cui, J. Widom, and J. L. Wiener. Tracing the lineage of view data in a warehousing environment. *ACM Trans. Database Syst.*, 25(2):179–227, 2000.

[10] Exabeam. Exabeam security information and event management. https://www.exabeam.com/product/siem/, 2025.

[11] V. B. Gadicha, A. B. Gadicha, M. Zuhair, Z. I. Khan, and M. S. Burange. Revolutionizing security information and event management (siem) systems: Harnessing deep learning for advanced threat detection. In *Project Management Information Systems: Empowering Decision Making and Execution*, pages 233–262. IGI Global Scientific Publishing, 2025.

[12] T. J. Green, G. Karvounarakis, and V. Tannen. Provenance semirings. In *Proc. PODS 2007*, pages 31–40, 2007.

[13] S. Hallé and H. Tremblay. A theory of fine-grained lineage for functions on structured objects. *Theor. Comput. Sci.*, 1039:115192, 2025.

[14] IBM. Ibm security qradar siem. https://www.ibm.com/products/qradar-siem, 2025. Retrieved September 25, 2025.

[15] W. Inc. The open source security platform. https://wazuh.com/, 2025. Retrieved September 25, 2025.

[16] N. F. Johnson and S. Katzenbeisser. A survey of steganographic techniques. In *Information Hiding*. Springer, 2000.

[17] H. Karlzén. An analysis of security information and event management systems-the use or siems for log collection, management and analysis. 2009.

[18] K. Kaur, I. Gupta, and A. K. Singh. Data leakage prevention: e-mail protection via gateway. In *Journal of Physics: Conference Series*, volume 933, page 012013. IOP Publishing, 2017.

[19] P. Lacharme and Y. Moy. Polyglot files: A journey into file format chimeras. In *Black Hat Europe*, 2018.

[20] W. Li, Y. Yan, H. Tu, and J. Xu. A dynamic taint tracking based method to detect sensitive information leaking. In *The 16th Asia-Pacific Network Operations and Management Symposium*, pages 1–4. IEEE, 2014.

[21] LogRhythm. Logrhythm siem self-hosted siem platform. https://logrhythm.com/products/logrhythm-siem/, 2025. Retrieved September 25, 2025.

[22] Y. Lu, X. Huang, Y. Ma, and M. Ma. A weighted context graph model for fast data leak detection. In *2018 IEEE International Conference on Communications (ICC)*, pages 1–6. IEEE, 2018.

[23] S. McCamant and M. D. Ernst. Quantitative information flow as network flow capacity. In *Proc. PLDI 2008*, pages 193–205, 2008.

[24] M. Meli, M. R. McNiece, and B. Reaves. How bad can it git? characterizing secret leakage in public github repositories. In *NDSS*, 2019.

[25] E. NV. Meet the search platform that helps you search, solve, and succeed. https://www.elastic.co/elastic-stack/, 2023. Retrieved July 22, 2023.

[26] F. A. Petitcolas, R. Anderson, and M. G. Kuhn. Information hiding—a survey. *Proceedings of the IEEE*, 87(7):1062–1078, 1999.

[27] S. Raman, A. Somayaji, and M. Mannan. Understanding data leak prevention. In *ASIACCS*, 2011.

[28] A. Sachenko, M. Vizhevskyi, et al. Modern strategies for data leak detection and prevention in dynamic environments. In *CEUR Workshop Proceedings*, 2025.

[29] Y. Shapira, B. Shapira, and A. Shabtai. Content-based data leakage detection using extended fingerprinting. *arXiv preprint arXiv:1302.2028*, 2013.

[30] X. Shu, D. Yao, and E. Bertino. Privacy-preserving detection of sensitive data exposure. *IEEE transactions on information forensics and security*, 10(5):1092–1103, 2015.

[31] Wazuh Inc. *Wazuh Security Platform Documentation*, 2023.

[32] Y. Xiaohui, S. Yubo, C. Fei, et al. Android's sensitive data leakage detection based on api monitoring. In *International Conference on Cyberspace Technology (CCT 2014)*, pages 1–4. IET, 2014.

[33] T. Yang, H. Cui, S. Niu, and P. Zhang. An analysis on sensitive data passive leakage in android applications. In *2015 IEEE 16th International Conference on Communication Technology (ICCT)*, pages 125–131. IEEE, 2015.

[34] D. Zhu, J. Jung, D. Song, T. Kohno, and D. Wetherall. Tainteraser: Protecting sensitive data leaks using application-level taint tracking. *ACM SIGOPS Operating Systems Review*, 45(1):142–154, 2011.